

Lab-1

Function creation for the water Jug problem
Lab-1 → State space of the water Jug problem

1-1 → def successors (state):

Defines a function named 'successors':
it takes an input called 'state'.

2-1 → $n, y = \text{state}$ ~~##~~ A state is represented as (n, y) where

$n =$ water in Jug x having 4L capacity
 $y =$ water in Jug y having 3L capacity

3-1 → $\text{pour_x_to_y} = \min(n, 3-y)$

~~##~~ It pours water from x to y such that
checking how much water can be poured from
 x to y .

→ It includes two limits ' n ' and ' y '
such that ' n ' cannot have more than n and
 y cannot have more than ' $3-y$ ', that is,
its remaining space.

4-1 → .) Then we will take the minimum value
out of both n and y .

4-2 → $\text{pour_y_to_x} = \min(y, 4-n)$

//_

2) Same it will do like "line-3" with one change that the water is poured from "y" to "x" Jug

3) → Return all the possible operations →

Return []

$(x, y) \rightarrow$ It is returning filled Jug x

$(x, 0) \rightarrow$ It is returning filled Jug y

$(0, y) \rightarrow$ It is returning empty Jug x

$(x, 0) \rightarrow$ It is returning empty Jug y

$(x - \text{pour } x \text{ to } y, y + \text{pour } x \text{ to } y) \rightarrow$ It is returning the poured water from Jug x to Jug y.

$(x + \text{pour } y \text{ to } x, y - \text{pour } y \text{ to } x) \rightarrow$

It is returning the poured water from Jug y to Jug x.

5) start = (0, 0)

→ Initially Jug x and Jug y has 0 of water

7) seen = {start}

→ This creates a set in which the state which is already visited or seen.

8) frontier = [start]

→ It contains the state which is to be explored

red in the current level.

8) level = 0

-) Current BFS level is 0 and it starts from there

9) while frontier and level < P:

-) This while loop will check for states less than till P

-) Only while loop is used because we don't know how many levels in BFS are there beforehand

-) And the limit of P is given because the while loop should not get trapped in infinite loop

10) new = []

-) It's an empty list which contains all newly discovered states

11) for s in frontier:

-) It takes one state from the current level.

12) for na in successors(s):

-) It generates all 6 possible next states

11

14) if not seen:

·) If the state has not been visited, continue otherwise we will ignore

·) It prevents duplicate exploration.

15) seen.add(ch)

·) It stores the new state forever

16) new.append(ch)

·) store it for the next BFS level

17) if h2[i] == 2:

·) it checks whether Jug x contains 2 litres

18) print("Goal: 2L in x at level", level+1, "x")

·) It prints goal message when Jug x has exactly 2L

19) level += 1

·) It increases the BFS level.

20) print("level", level, "→", len(new), "new states", sorted(new))

·) It prints the current level, no. of newly discovered states, and all new states in sorted order

//_

Frontier = new

- The newly discovered states become the next level to explore.

```
print("Total states found : ", len(Seen))
```

- Prints the total no. of unique reachable states.